Network Security

Network Security Report

GitHub Link : https://github.com/dev-69/secure-chat-laughing-engine

Phase 1 — Baseline Chat Application

1. System Overview

Phase 1 implements a basic one-to-one chat application using TCP sockets. The system consists of two programs:

- `server.cpp` — handles client connections and forwards messages.
- `client.cpp` — provides the interface used by each client.

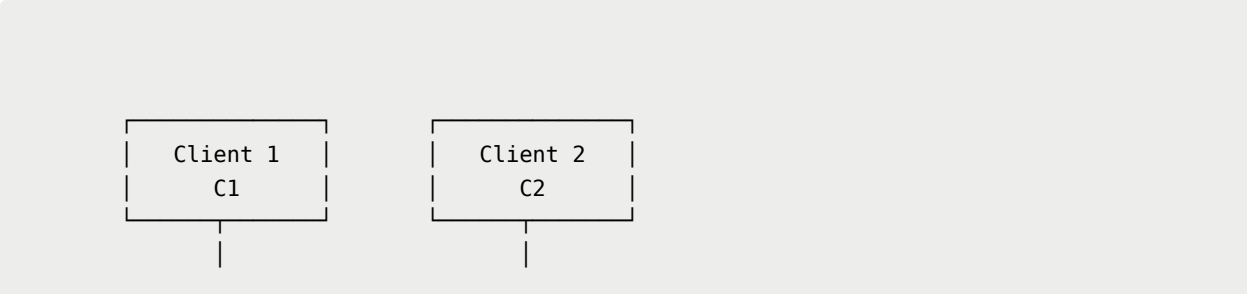
The server listens for TCP connections on **port 1111**. Two clients can connect to the server at the same time and communicate through it.

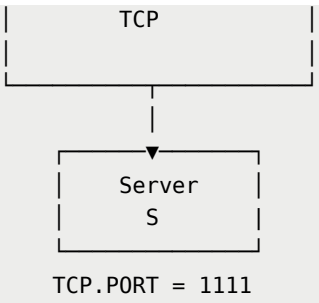
The application was tested using **three separate physical laptops connected to the same network**. One laptop was used as the server and the other two were used as clients.

---

2. System Architecture

The communication follows the architecture below:





The server keeps track of connected clients and forwards each message to the intended recipient.

---

3. Server Implementation

The server creates a TCP socket, binds it to port `1111`, and starts listening for incoming connections.

Each new client connection is handled using a separate thread:

```
std::thread clientThread(handleClient, clientSocket);
clientThread.detach();
```

This allows multiple clients to stay connected and communicate concurrently.

The server maintains a mapping between usernames and socket descriptors:

```
std::unordered_map<std::string, int> clients;
```

For example:

```
client1 → socket descriptor
client2 → socket descriptor
```

Since multiple threads can access this mapping, a mutex is used to protect it from concurrent access.

---

4. Message Routing

Messages can be sent using the following format:

```
@username message
```

For example:

```
@client2 Hello from client1
```

The client extracts the destination username and sends the message to the server. The server looks up that username in its client map and forwards the message to the corresponding socket.

The server also prints successfully relayed messages. For example:

```
[CHAT] client1 -> client2 : Hello from client1
```

This is also useful for demonstrating the security problem in Phase 1: the server has direct access to the complete message because the message is still plaintext.

---

5. Client Commands

The client supports the required commands:

```
@username message

/chat username

/who

/quit
```

`/chat`

Sets the current chat partner so that the username does not need to be specified for every message.

`/who`

Requests the list of currently connected users from the server.

`/quit`

Terminates the client connection cleanly.

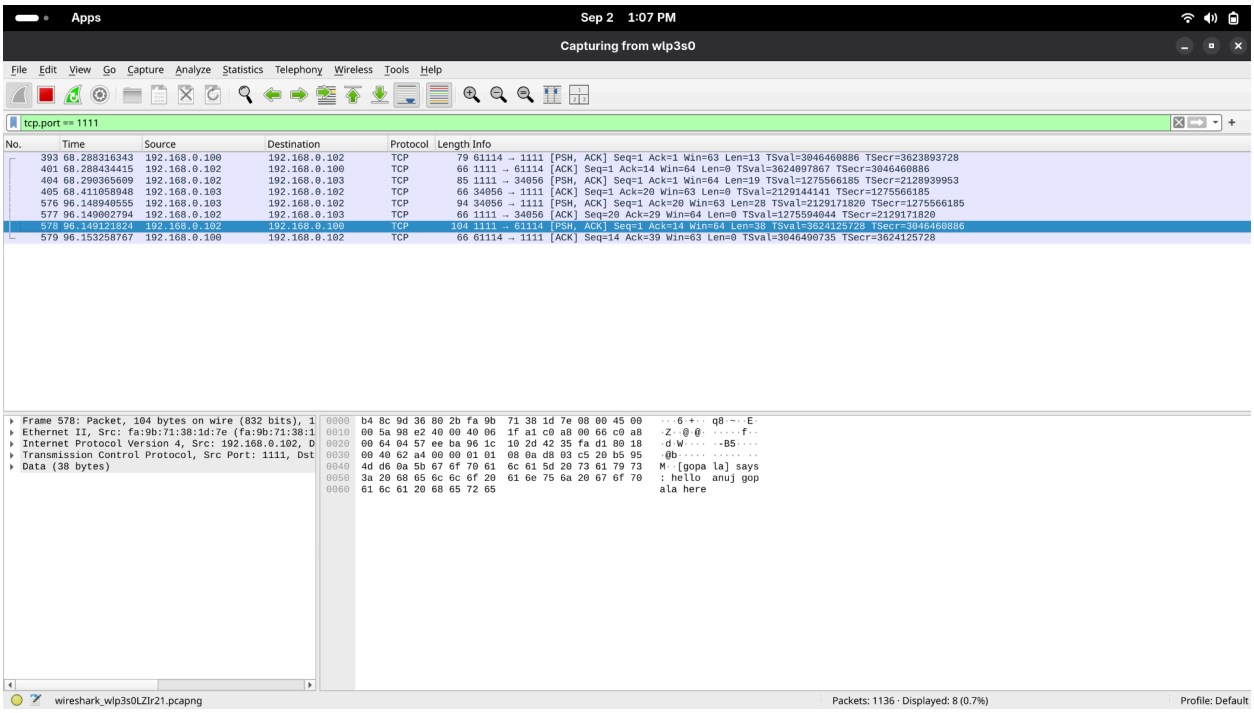
---

6. Plaintext Verification

No encryption is used in Phase 1. Messages are therefore transmitted directly over the TCP connection.

The server logs already show the message contents in plaintext. To verify that the same information is visible on the network, traffic was captured using **Wireshark**.

Figure 1 – Wireshark TCP Stream Showing Plaintext Communication



The capture confirms that someone monitoring the network traffic can read the exchanged messages without needing any decryption.

Phase 2 — Client-Server Confidentiality Using Diffie-Hellman

1. Overview

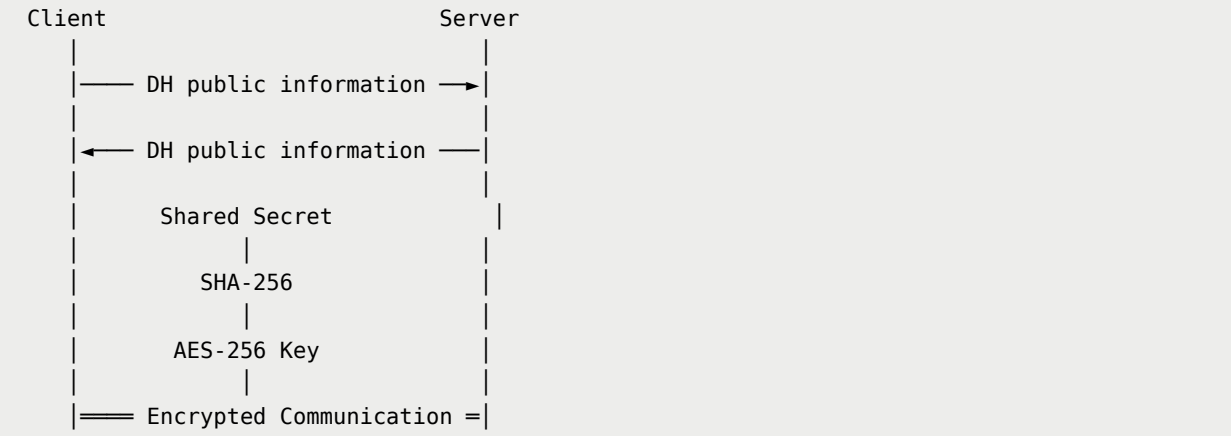
Phase 2 adds confidentiality to the Phase 1 chat application.

A manually implemented **Diffie-Hellman (DH)** key exchange is used to establish a shared secret between each client and the server. The shared secret is then processed using **SHA-256** to derive a 256-bit AES key.

Messages are subsequently encrypted using **AES-256-GCM**.

The DH implementation uses OpenSSL's low-level `BIGNUM` operations and modular exponentiation rather than a built-in DH/ECDH implementation.

The basic flow is:



The same process is performed independently for every client.

---

2. Diffie-Hellman Implementation

The DH functionality is implemented in `dh.cpp`.

The implementation uses the **RFC 3526 2048-bit MODP Group (Group 14)**.

The generator is set to:

```
BN_set_word(g, 2);
```

A private value is generated for each `DHE` instance:

```
BN_rand_range(priv_key, p);
```

The corresponding public value is calculated using modular exponentiation:

```
BN_mod_exp(pub_key, g, priv_key, p, ctx);
```

For a client and server, the exchange can be represented as:

```
Client:

private = a
public  = g^a mod p

Server:

private = b
public  = g^b mod p
```

The client computes:

```
(g^b)^a mod p
```

while the server computes:

```
(g^a)^b mod p
```

Both calculations produce the same shared secret.

The secret itself is never transmitted over the network.

---

3. Independent Key for Each Client

The server creates a separate `DHE` object for every client connection:

```
void handleClient(int clientSocket) {

    DHE dh;

    dh.generateKeys();
```

```
    ...

}
```

Since `handleClient()` is executed separately for each client, every client performs its own DH exchange with the server.

The server also stores the corresponding `DHE` object along with the client's socket:

```
struct ClientData {

    int socket;

    DHE* dh_ptr;

};
```

This allows the server to use the correct encryption key for each connection.

Conceptually:

```
Client 1 ———— Key K1 ———— Server

Client 2 ———— Key K2 ———— Server

                  K1 ≠ K2
```

Thus, the two client connections do not share the same symmetric encryption key.

---

4. Deriving the AES Key

The raw DH shared secret is not directly used as the AES key.

After calculating the shared secret, it is converted into bytes and hashed using SHA-256:

```cpp
EVP_DigestInit_ex(mdctx, EVP_sha256(), nullptr);

EVP_DigestUpdate(
    mdctx,
    secret_bytes.data(),
    secret_bytes.size()
);

EVP_DigestFinal_ex(
    mdctx,
    aes_key.data(),
    &hash_len
);
```

SHA-256 produces 32 bytes:

```
32 bytes × 8 = 256 bits
```

The resulting value is therefore used as the AES-256 key.

The overall process is:

```
DH Shared Secret
       |
       ▼
    SHA-256
       |
       ▼
  256-bit AES Key
```

Using SHA-256 also gives a fixed-length representation of the DH shared secret that can be directly used as a symmetric key.

---

5. AES-GCM Encryption

Once the DH exchange is complete, communication is encrypted using **AES-256-GCM**.

A new 12-byte random nonce is generated for every message:

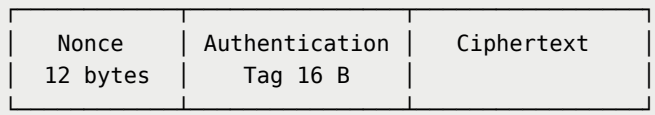
```
unsigned char nonce[12];

RAND_bytes(nonce, sizeof(nonce));
```

AES-256-GCM is then initialized using the derived key and nonce:

```
EVP_EncryptInit_ex(
    ctx,
    EVP_aes_256_gcm(),
    nullptr,
    aes_key.data(),
    nonce
);
```

The encrypted message contains the nonce, authentication tag, and ciphertext:



The GCM authentication tag allows the receiver to detect if the encrypted data has been modified.

Therefore, AES-GCM provides both:

- Confidentiality
- Integrity/authentication of the ciphertext

---

6. Encrypted Login and Chat Communication

The username exchange is also protected after the DH handshake.

For example, the server encrypts its username prompt:

```
std::vector<unsigned char> enc_prompt = dh.encrypt(prompt);
```

The client decrypts the prompt using the derived AES key.

The username entered by the client is also encrypted before being sent:

```
std::vector<unsigned char> enc_username = dh.encrypt(usernam
e);
```

Normal application messages and commands are also encrypted, including:

```
@client2 Hello

/who

/quit
```

Therefore, after the initial DH exchange, the actual application data is not transmitted as plaintext.

---

7. DH Fingerprint Verification

A fingerprint is generated from the derived AES key to make it easier to verify that both sides obtained the same key.

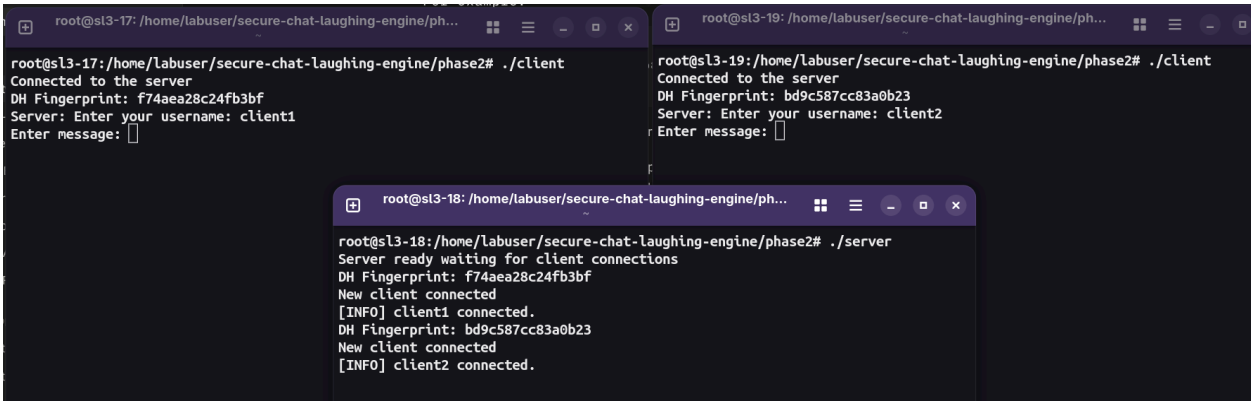
The fingerprint is produced by hashing the AES key and displaying the first eight bytes of the resulting hash.

The actual shared secret and AES key are never printed.

During testing, the fingerprint displayed by the client matched the fingerprint displayed by the server for the corresponding connection.

This provides evidence that both endpoints independently derived the same symmetric key.

Figure 2 — Matching DH Fingerprints



---

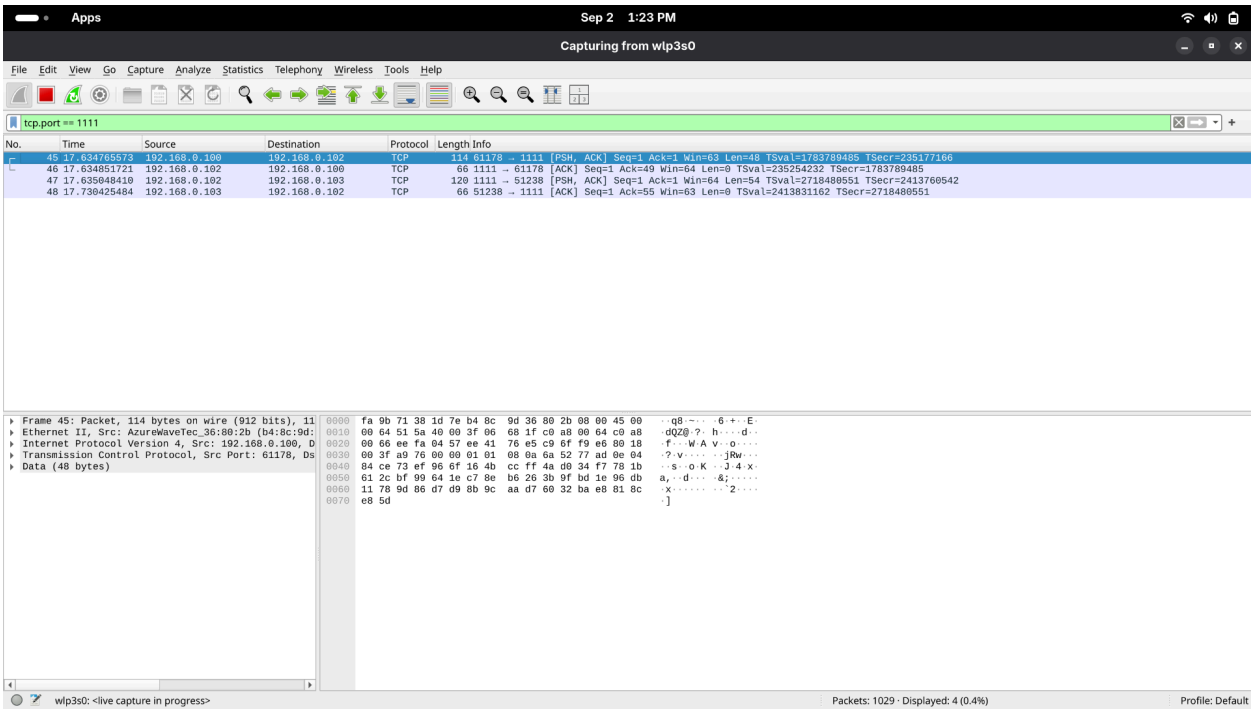
8. Wireshark Verification

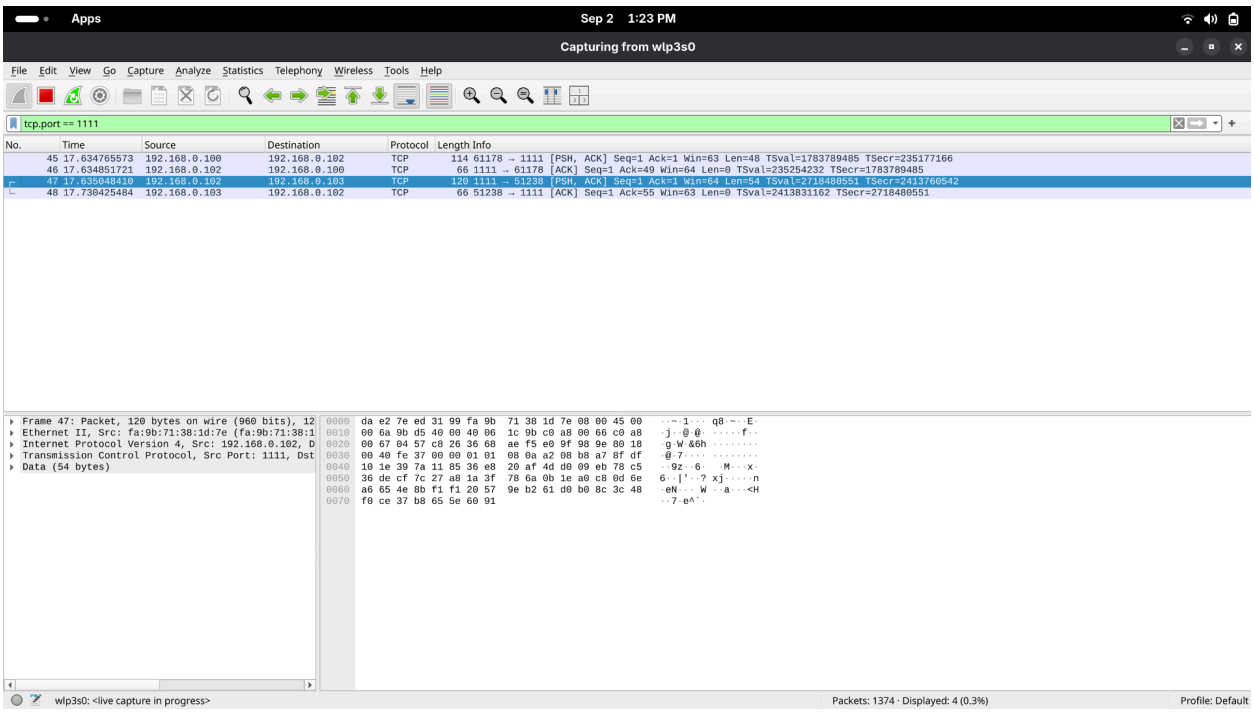
The Phase 1 traffic capture was repeated after adding encryption.

The traffic was captured using Wireshark on TCP port `1111` and inspected using:

In Phase 2, the application data appears as binary-looking encrypted data rather than readable chat messages.

Figure 3 — Wireshark Showing Encrypted Phase 2 Traffic





This shows that a passive observer monitoring the network can no longer directly read the chat messages from the captured TCP traffic.

9. AES-GCM Tamper Detection

AES-GCM also protects the integrity of the encrypted message.

To test this, one byte of the ciphertext was modified before attempting to decrypt it.

The decryption process verifies the GCM authentication tag:

```
EVP_CIPHER_CTX_ctrl(
    ctx,
    EVP_CTRL_GCM_SET_TAG,
    16,
    tag
);
```

If the ciphertext has been modified, authentication fails and the message is rejected:

```
if (ret <= 0) {
    return "<DECRYPTION_FAILED_TAMPERING_DETECTED>";
}
```

The observed output was:

```
<DECRYPTION_FAILED_TAMPERING_DETECTED>
```

Figure 4 — AES-GCM Tampering Test

The test confirms that modification of the encrypted data is detected rather than being silently accepted.

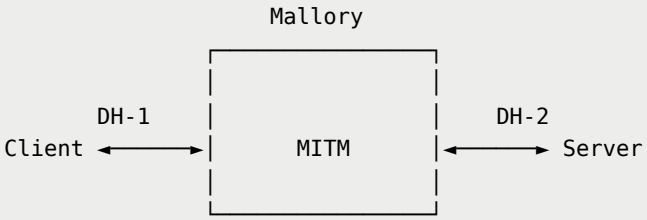
---

10. Man-in-the-Middle Attack

Although DH protects against passive eavesdropping, the basic DH exchange does not authenticate who is participating in the exchange.

To demonstrate this limitation, a separate `mitm.cpp` proxy was implemented.

The proxy performs two independent DH exchanges:



This results in two different keys:

```
Client ↔ Mallory : Key K1

Mallory ↔ Server : Key K2
```

The client believes it has established a secure connection with the server, while the server believes it has established a secure connection with the client.

Mallory, however, has access to both keys.

---

11. MITM Message Interception

The proxy decrypts messages received from the client using the client-facing key:

```
std::string plaintext =
    dh_client->decrypt(payload);
```

The decrypted message is then logged:

```
std::cout << "\n[INTERCEPT C->S]: "
          << plaintext << std::endl;
```

Mallory then encrypts the plaintext using the server-facing key and forwards it to the real server.

The same process happens for messages travelling from the server back to the client.

The attack can therefore be represented as:

```
Client
  |
  | Encrypted with K1
  ▼
Mallory
  |
  | Decrypt
  |
  | Plaintext
  |
  | Encrypt with K2
  ▼
Server
```

Figure 5 — MITM Proxy Intercepting Plaintext

```
dibyansh@LAPTOP-3AF6P4N9:/mnt/c/Users/HP/Desktop/Assignments/dev - network/secure-chat-laughing-engine/phase2$ ./client
Connected to the server
DH Fingerprint: 436dca84a9403d70
Server: Enter your username: dibyansh_gupta
Enter message: /who
Enter message:

--- Online Users ---
- dibyansh_gupta
- anuj_ramane
--------------------

/chat anuj_ramane
Chat partner set to anuj_ramane
Enter message: hello anuj how are you
Enter message:
```

```
75      DHE dh_server_facing;
76      DHE dh_client_facing;

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS        ./server - phase2

hedwig69@fedora:~/Desktop/IIT-Bombaee/Semester3/CS6008 - Network Security/Secure_Chat_Assignment/phase2$ ./server
Server ready waiting for client connections
DH Fingerprint: 436dca84a9403d70
New client connected
DH Fingerprint: 77ad40861c2dcfee
New client connected
[INFO] anuj_ramane connected.
[INFO] dibyansh_gupta connected.
[CHAT] dibyansh_gupta -> anuj_ramane : hello anuj how are you
```

```
105
106     std::thread t1(relay_client_to_server, client_fd, server_fd, &dh_client_facing, &dh_server_facing);

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS        ./mitm - phase2

gopala@fedora:~/Desktop/Dev/secure-chat-laughing-engine/phase2$ ./
client         client_normal  client_victim  mitm          server
gopala@fedora:~/Desktop/Dev/secure-chat-laughing-engine/phase2$ ./mitm
[MITM] Connected to real Server.
[MITM] Waiting for victim client on port 9999...
[MITM] Victim Client connected!
[MITM] Handshakes complete. Logging traffic...

[INTERCEPT S->C]: Server: Enter your username:

[INTERCEPT C->S]: anuj_ramane

[INTERCEPT S->C]:
[dibyansh_gupta] says: hello anuj how are you
```

```
Problems 6  Output  Debug Console  Terminal  Ports        wsl - phas

anujr@AnujASUS:/mnt/c/Users/anujr/Downloads/dev/secure-chat-laughing-engine/phase2$ g++ client.cpp -o client -lcrypto
anujr@AnujASUS:/mnt/c/Users/anujr/Downloads/dev/secure-chat-laughing-engine/phase2$ ./client
Connected to the server
DH Fingerprint: 40d26dba87280c38
Server: Enter your username: anuj_ramane
Enter message:

[dibyansh_gupta] says: hello anuj how are you
```

This shows that unauthenticated DH alone is vulnerable to an active MITM attack.

---

12. Phase 2 Result

Phase 2 adds confidentiality and authenticated encryption to the chat application.

Each client independently performs a DH exchange with the server and derives its own AES-256 key. Messages are then protected using AES-256-GCM.

The following tests were performed:

- Matching DH fingerprints on both endpoints
- Wireshark inspection of encrypted traffic
- AES-GCM ciphertext tampering
- MITM interception using the Phase 2 proxy

The Wireshark capture confirms that the application messages are no longer visible as plaintext, and the tampering test shows that modified ciphertext is rejected.

However, the MITM experiment exposes a limitation of plain DH: **the protocol establishes a shared secret but does not prove the identity of the other party**.

This is the problem addressed in Phase 3.

Phase 3 — Server Authentication Using PKI

Objective

Phase 3 adds server authentication using **Public Key Infrastructure (PKI)**.

The client now verifies that it is communicating with the legitimate server before continuing with the Diffie-Hellman exchange.

The authentication process has two parts:

1. Certificate validation
2. Proof-of-possession of the server's private key

Only after both checks succeed does the client continue with the DH exchange.

---

1. Certificate Authority and Server Certificate Setup

A private Certificate Authority was created using OpenSSL.

The CA consists of:

- A private CA key
- A self-signed CA certificate

A separate server key pair was then generated. A Certificate Signing Request (CSR) was created from the server public key and signed by the CA.

The resulting files are:

```
certs/

├── ca.key
├── ca.crt
├── server.key
└── server.crt
```

The following commands were used.

Generate the CA private key

```
openssl genrsa -out certs/ca.key 4096
```

Generate the self-signed CA certificate

```
openssl req -x509 -new -nodes \
    -key certs/ca.key \
    -sha256 -days 3650 \
    -out certs/ca.crt \
    -subj "/C=IN/ST=Maharashtra/L=Mumbai/O=SecureChat/OU=CA/C
N=SecureChat Root CA"
```

Generate the server private key

```
openssl genrsa -out certs/server.key 2048
```

Generate the server CSR

```
openssl req -new \
    -key certs/server.key \
    -out certs/server.csr \
    -subj "/C=IN/ST=Maharashtra/L=Mumbai/O=SecureChat/OU=Serv
er/CN=SecureChat Server"
```

Sign the server CSR using the CA

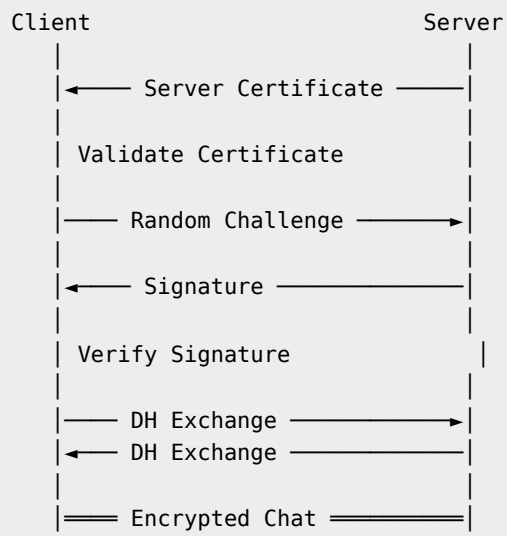
```
openssl x509 -req \
    -in certs/server.csr \
    -CA certs/ca.crt \
    -CAkey certs/ca.key \
    -CAcreateserial \
    -out certs/server.crt \
```

```
    -days 365 \
    -sha256
```

2. Updated Client and Server

The Phase 2 client and server were modified to add certificate authentication before the DH exchange.

The new handshake is:



2.1 Certificate Validation

Before performing DH, the server sends its certificate to the client.

The client receives the certificate and validates it against the trusted CA certificate:

```
certs/ca.crt
```

The client checks that the certificate is:

- Signed by the trusted CA
- Within its validity period
- Valid according to the configured trust store

If certificate validation fails, the client closes the connection and does not continue with the DH exchange.

This prevents an attacker from simply presenting an arbitrary certificate to the client.

2.2 Proof-of-Possession

Certificate validation alone is not sufficient.

An attacker could potentially obtain a copy of the legitimate server certificate. However, the attacker should not be able to authenticate without the corresponding private key.

To test this, the client generates a random 32-byte challenge:

```
Client → Server : Random Challenge
```

The legitimate server signs the challenge using:

```
certs/server.key
```

The signature is sent back to the client:

```
Client → Server : Challenge

Client ← Server : Signature(Challenge)
```

The client extracts the public key from the already-validated server certificate and uses it to verify the signature.

Only if the signature verification succeeds does the client continue with the DH exchange.

In other words:

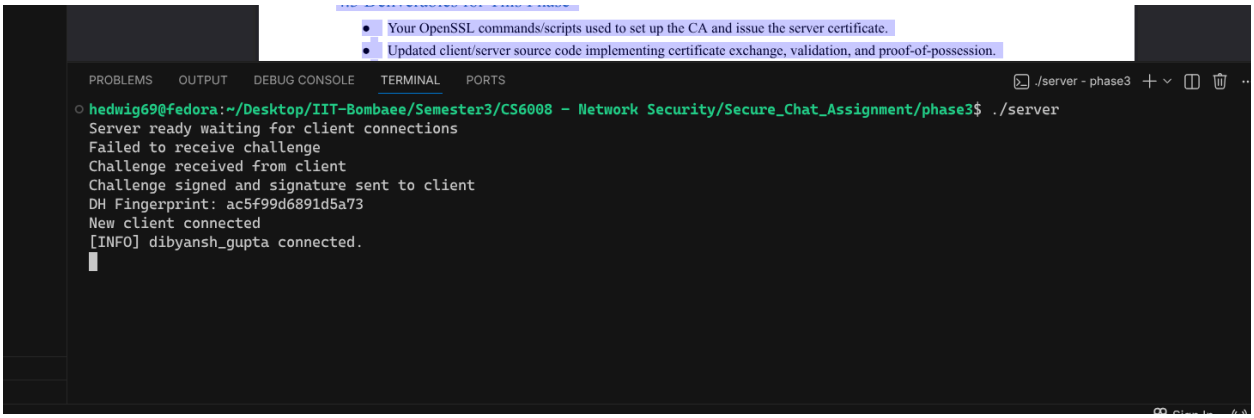
```
Certificate
     |
     ▼
Is this certificate trusted?
     |
     ▼
Challenge
     |
     ▼
Can the server produce a valid signature?
     |
     ▼
Yes
     |
     ▼
Continue with DH
```

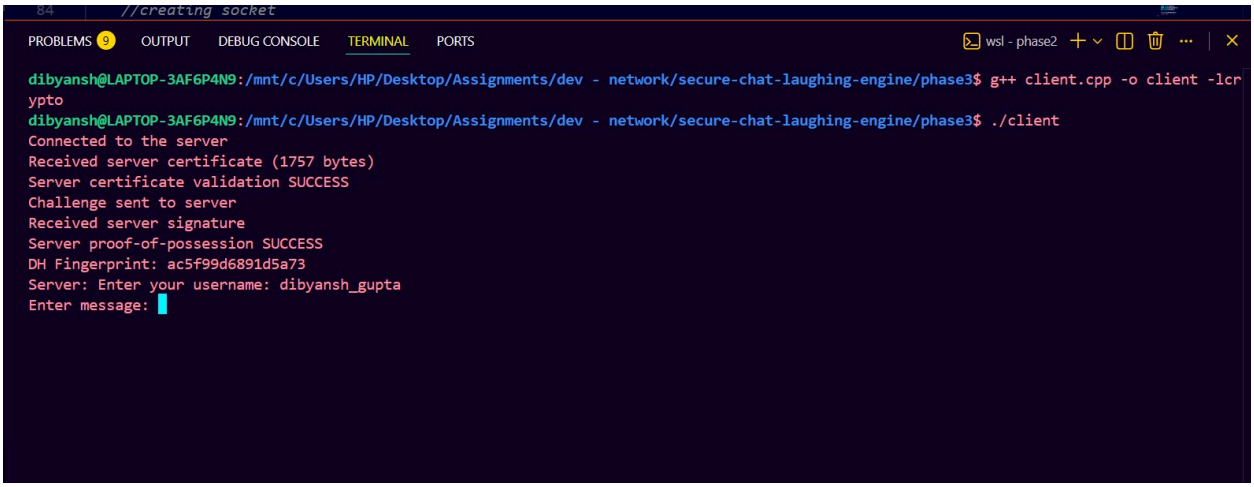
2.3 Legitimate Phase 3 Connection

A legitimate connection was tested using the real server certificate and corresponding private key.

The client output showed successful certificate validation and proof-of-possession before the DH exchange continued.

Figure 7 — Legitimate Phase 3 Connection





The screenshot should show output similar to:

```
Received server certificate

Server certificate validation SUCCESS

Challenge sent to server

Received server signature

Server proof-of-possession SUCCESS

DH Fingerprint: ...
```

---

3. Updated MITM Attack

The MITM proxy from Phase 2 was modified to attack the Phase 3 protocol.

In the first attack, Mallory generates a different certificate that is **not signed by the trusted CA**.

Mallory sends this certificate to the victim client.

The client attempts to validate the certificate using:

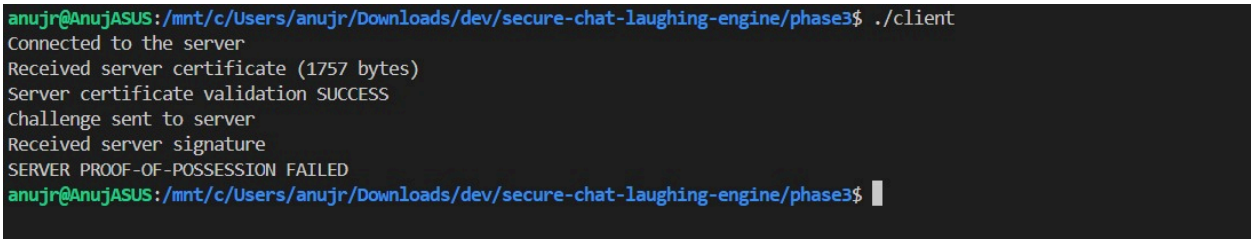
```
certs/ca.crt
```

Since Mallory's certificate was not issued by the trusted CA, certificate validation fails.

The client then closes the connection before the DH exchange can take place.

The expected behaviour is:

Figure 8 — MITM Attack Fails Due to Invalid Certificate



---

4. Proof-of-Possession Bypass Attempt

A second attack was performed to test whether an attacker could bypass authentication simply by obtaining a copy of the legitimate server certificate.

In this attack, Mallory possesses:

```
server.crt
```

but does **not** possess:

```
server.key
```

Mallory forwards the legitimate server certificate to the client.

Because the certificate is valid and trusted, the certificate validation step succeeds.

The client then generates a random challenge.

Mallory attempts to sign the challenge using a different private key.

The client verifies the received signature using the public key contained in the legitimate server certificate.

Since Mallory's private key does not correspond to the public key in the certificate, verification fails.

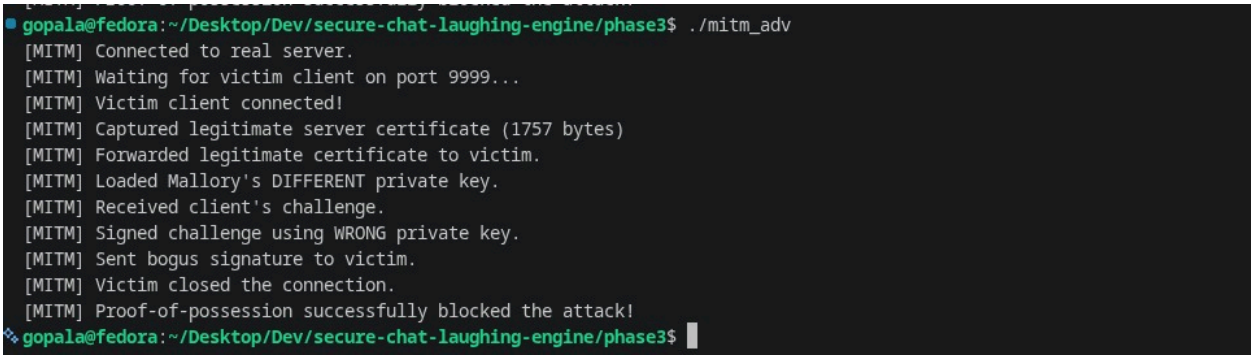
The client therefore rejects the connection.

Expected output:

```
Received server signature

SERVER PROOF-OF-POSSESSION FAILED
```

Figure 9 — Proof-of-Possession Bypass Attempt Rejected

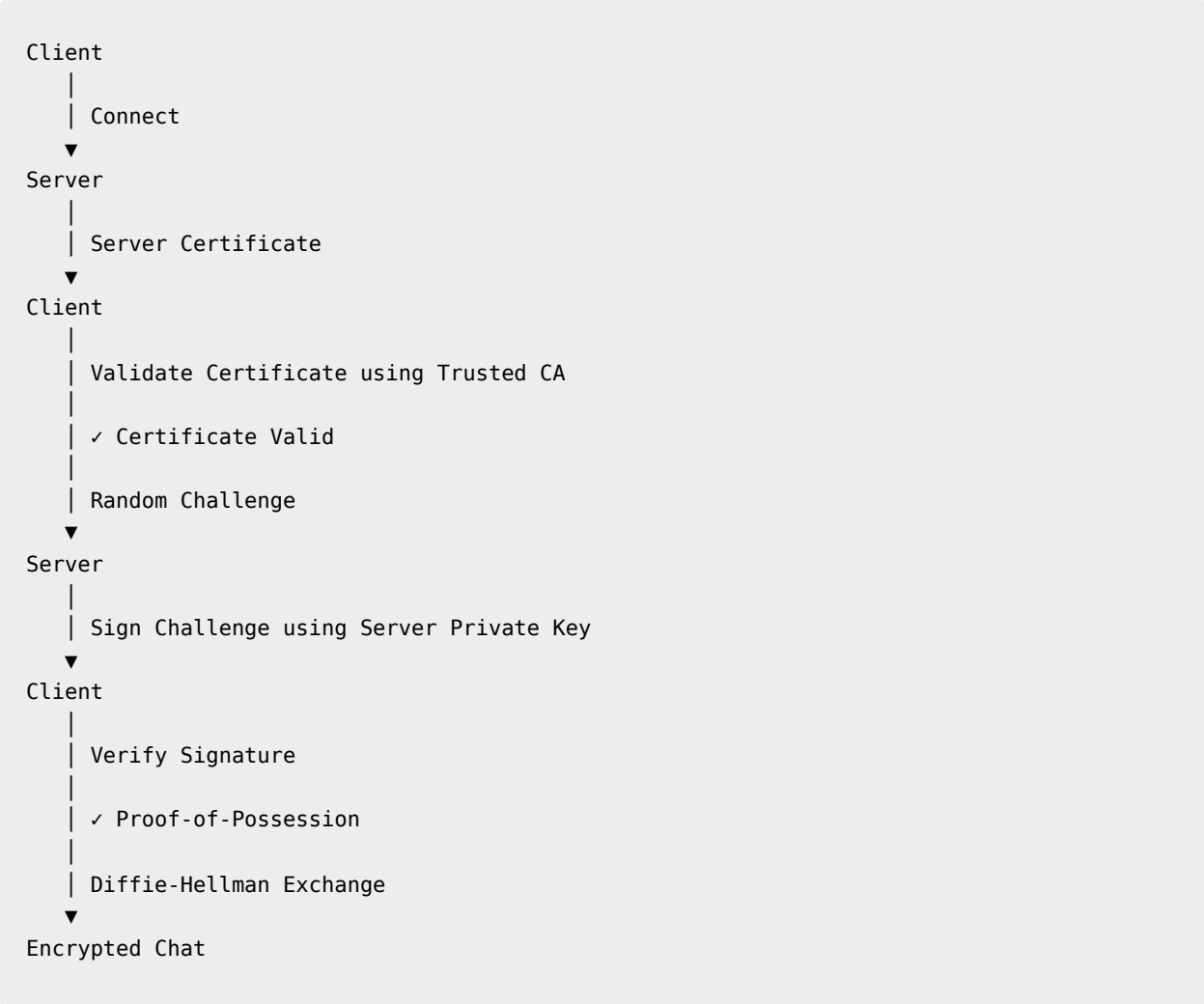


This test is important because it shows that simply possessing a valid certificate is not enough. The server must also demonstrate possession of the corresponding private key.

---

5. Phase 3 Security Flow

The final Phase 3 handshake can be summarized as follows:



The important difference from Phase 2 is that the DH exchange is no longer performed with an unauthenticated peer.

---

6. Phase 3 Result

Phase 3 adds server authentication to the secure chat application using PKI.

The client now performs two authentication checks before establishing the encrypted session:

1. **Certificate validation** — verifies that the server certificate was issued by the trusted CA.
2. **Proof-of-possession** — verifies that the server actually possesses the private key corresponding to the certificate.

The legitimate server passes both checks and is then allowed to proceed with the DH exchange.

The modified MITM proxy was tested in two ways:

- Using an untrusted certificate — rejected during certificate validation.
- Using the legitimate certificate without the corresponding private key — rejected during proof-of-possession.

Therefore, the Phase 2 MITM attack is no longer successful against the Phase 3 authentication mechanism.

The main security improvement introduced in this phase is that the client can now **authenticate the server before trusting the DH key exchange**.

Phase 4 — End-to-End Encryption Between Clients

1. Overview

Phase 4 adds an additional end-to-end encryption layer between clients.

When Client 1 executes:

```
/e2e client2
```

Client 1 and Client 2 perform a separate Diffie-Hellman exchange directly at the application level. The server only relays the

handshake messages and does not obtain the resulting shared key.

Once the E2E session is established, chat messages are encrypted using the client-to-client key before being sent through the existing encrypted client-server connection from Phase 3.

The resulting structure is:

```
Client 1
   |
   | E2E encryption
   ▼
E2E Ciphertext
   |
   | Client-server encryption
   ▼
Server
   |
   | Relay only
   ▼
Client 2
   |
   | E2E decryption
   ▼
Plaintext
```

The server can still see routing information such as the sender and destination username, but it cannot derive the E2E key or decrypt the message content.

---

2. E2E Key Establishment

The `/e2e username` command triggers the client-to-client key exchange.

The protocol distinguishes E2E handshake messages from normal chat messages using tags such as:

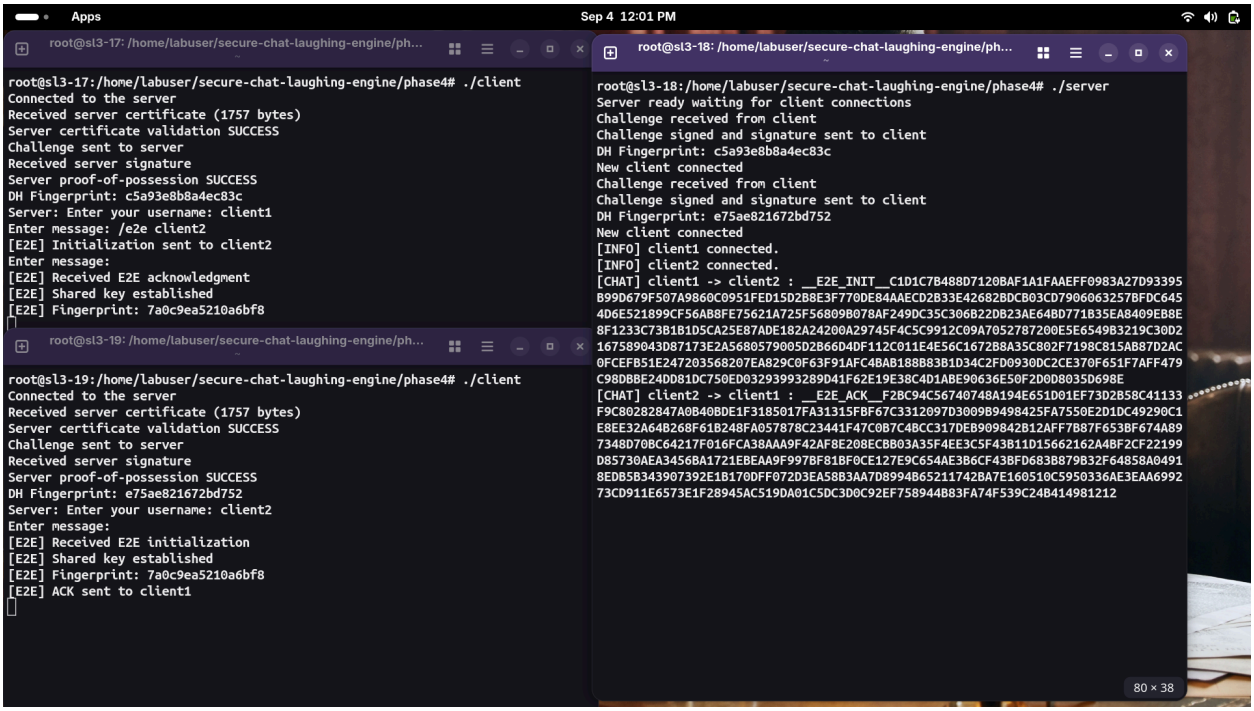
```
__E2E_INIT__
__E2E_ACK__
```

```
__E2E_MSG__
```

The server does not need to understand these messages cryptographically. It simply forwards them to the appropriate client.

After the exchange, both clients independently derive the same E2E key.

Figure 10 — Matching E2E Fingerprints



The matching fingerprints provide evidence that both clients derived the same E2E key.

---

3. End-to-End Message Encryption

After the E2E session is established, messages are encrypted using the client-to-client key and AES-GCM.

The resulting ciphertext is wrapped using:

```
__E2E_MSG__
```

and sent through the already encrypted client-server connection.

The server therefore receives only opaque E2E ciphertext.

Figure 11 — Server Relaying E2E Ciphertext

The server can identify that `client1` is sending data to `client2`, but the actual message content is not visible.

For comparison, before an E2E session is established, the server can still read normal chat messages, consistent with the Phase 2/3 design.

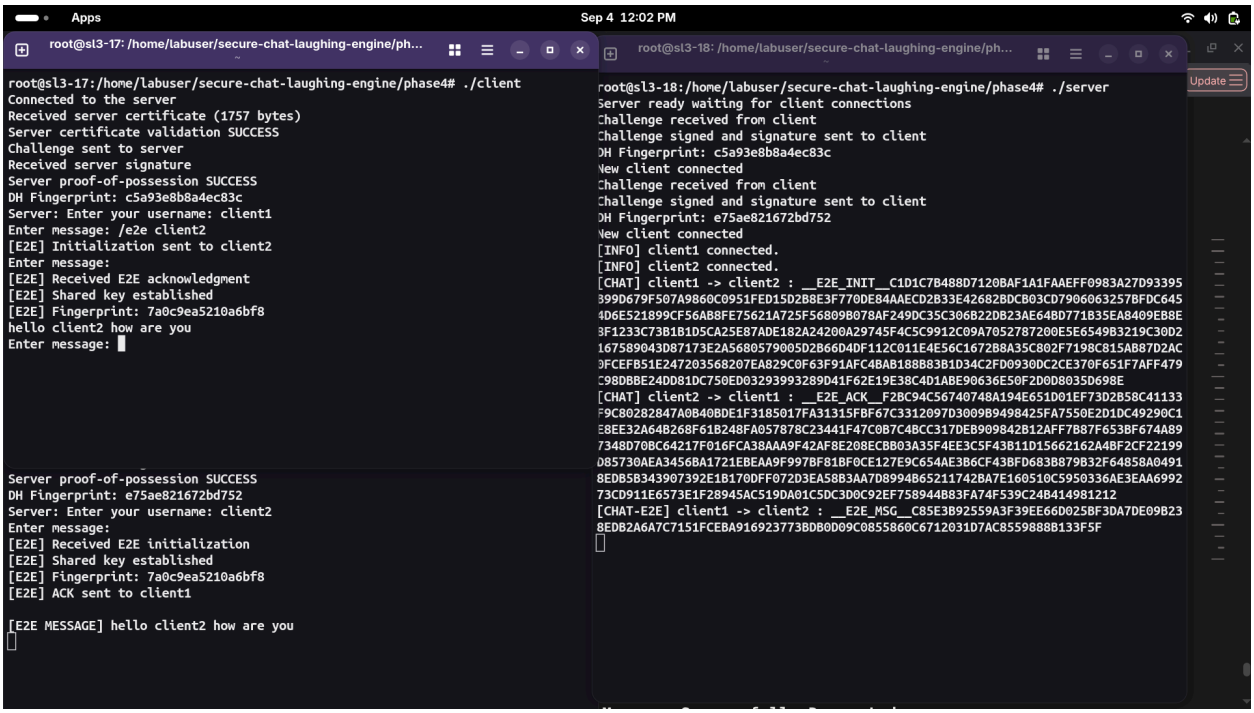
---

4. End-to-End Verification

A test message was sent from Client 1 to Client 2 after establishing the E2E session.

The server log showed only the E2E ciphertext, while Client 2 successfully decrypted and displayed the original message.

Figure 12 — E2E Message Successfully Decrypted



This confirms that:

- Client 1 and Client 2 successfully established a shared E2E key.
- The server could not read the E2E message content.
- The server continued to function purely as a relay.
- Client 2 could correctly decrypt the message.

---

5. Phase 4 Result

Phase 4 introduces an additional encryption layer between clients without replacing the existing client-server security from Phase 3.

The server continues to provide routing and transport, but it does not possess the client-to-client E2E key. The matching fingerprints demonstrate that the two clients independently derived the same key, while the server log demonstrates that E2E chat content is transmitted only as opaque ciphertext.

Thus, the confidentiality of messages is extended from client-server encryption to true client-to-client end-to-end encryption.

Phase 5 — Forward Secrecy via Key Rotation

1. Overview

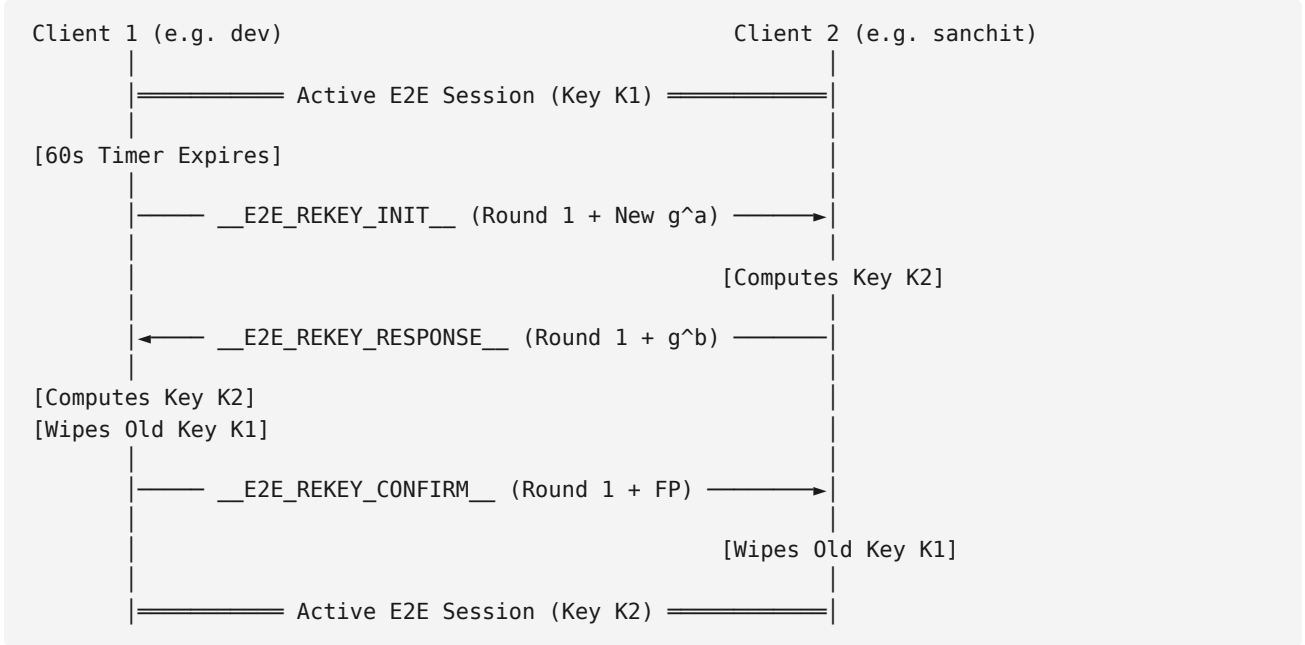
While Phase 4 provides end-to-end confidentiality between clients, it relies on a single, long-lived session key negotiated at connection time. If an adversary passively logs all encrypted network traffic over the wire and later compromises one of the endpoints (or extracts the key from memory), the adversary can retroactively decrypt all historical messages.

Phase 5 addresses this limitation by introducing **Forward Secrecy via periodic key rotation**. The shared symmetric key between Client 1 and Client 2 is automatically renegotiated on a fixed timer every 60 seconds. Each renegotiation produces a genuinely new, independent key, and the previous key is discarded and erased from memory.

---

2. Automatic Key Rotation Architecture

Key rotation is managed automatically by a dedicated background thread (`rekeyTimer`) running on each client:



For every rotation cycle:

1. A fresh ephemeral Diffie–Hellman key pair is generated using RFC 3526 Group 14 (`new DHE()`, `generateKeys()`).
2. The public key is exchanged inside an encrypted control message relayed through the server.
3. Both clients compute the new shared secret, hash it with SHA-256 to derive a new 256-bit AES key, and verify matching fingerprints.
4. The previous key is securely wiped and freed from memory using `destroyDHE()`.

---

3. Forward Secrecy and Old Key Destruction

To guarantee forward secrecy, old keys must not persist in process memory. The `destroyDHE` function explicitly clears the OpenSSL `BIGNUM` private and public components using `BN_clear()`, zeroes out the raw AES symmetric key vector with `std::fill`, and frees the allocated memory:

```
void destroyDHE(DHE*& dh) {
    if (dh != nullptr) {
        BN_clear(dh->priv_key);
        BN_clear(dh->pub_key);
        std::fill(dh->aes_key.begin(), dh->aes_key.end(), 0);
        delete dh;
        dh = nullptr;
    }
}
```

Once confirmation completes, the old session key is permanently erased from memory.

---

4. Collision Avoidance Mechanism

A critical challenge in decentralized rekeying is handling race conditions where both clients attempt to trigger renegotiation at approximately the same time. If both clients simultaneously send rekey initialization requests, they risk computing mismatched keys or deadlocking.

To resolve this, the protocol enforces a deterministic leader election based on lexicographical username comparison:

```
if (localUsername >= e2ePartner) {
    return; // Only the client with lower username initiates rotation
}
```

By designating the client with `localUsername < e2ePartner` (e.g. `dev < sanchit`, where `dev` initiates rotation) as the authoritative rotation initiator,

collisions are eliminated by design. If an unexpected rekey initialization arrives while one is already in progress, the lower-priority peer yields, drops its pending request, and answers the initiator's request.

---

5. Verification and Results

5.1 Successive Key Rotations (Fingerprints and Timestamps)

The chat application was deployed and verified across separate virtual machines:

- **Server VM (`server`):** `10.91.240.119:1111`
- **Client 1 VM (`client1`):** `10.91.240.136` (User: `dev`)
- **Client 2 VM (`client2`):** `10.91.240.179` (User: `sanchit`)

The verification demonstrated the following lifecycle across two full rotations:

- **Initial Handshake:** Both client VMs established the initial E2E session across the virtual network and printed matching fingerprints:

  ```
  [E2E] Fingerprint: 036f6ef30aa52e33
  ```

- **Pre-Rotation Chat:** Client 1 (`dev`) transmitted `"Hello sanchit how are you ?"`, which Client 2 (`sanchit`) received and decrypted successfully.
- **60-Second Rotation (Round 1):** After 60 seconds, the background timer automatically triggered rotation. Both clients renegotiated keys and confirmed matching fingerprints at the exact same timestamp:

  ```
  [E2E] Key rotation completed at 2026-09-04 23:23:58 (Round 1)
  [E2E] New fingerprint: c2d66cf834bb1a56
  ```

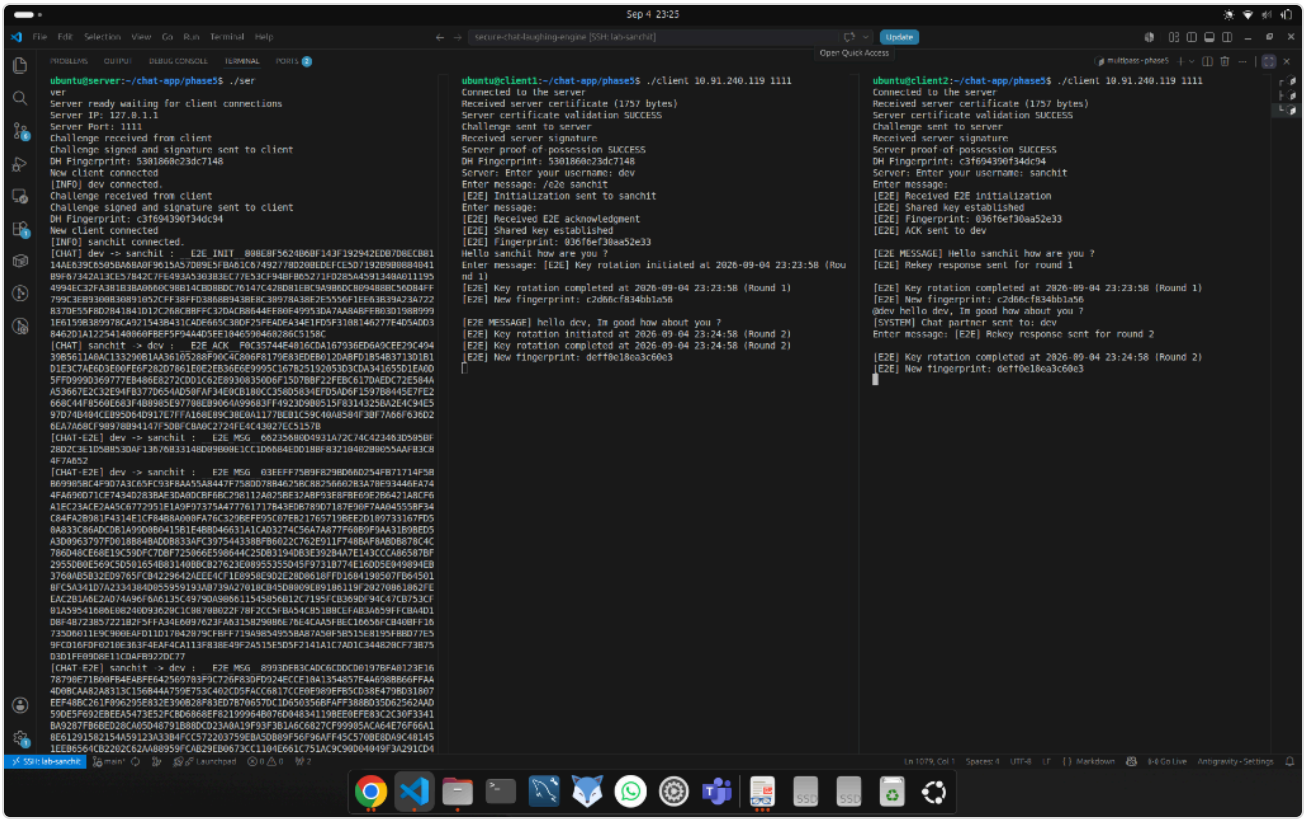
- **60-Second Rotation (Round 2):** After another 60 seconds, the background timer triggered a second rotation. Both clients renegotiated keys again and confirmed matching fingerprints at the exact same timestamp:

  ```
  [E2E] Key rotation completed at 2026-09-04 23:24:58 (Round 2)
  [E2E] New fingerprint: deff0e18ea3c60e3
  ```

The fingerprints changed from `036f6ef30aa52e33` to `c2d66cf834bb1a56` to `deff0e18ea3c60e3`, proving across two successive rotations that:

1. The fingerprint changes each time (genuinely fresh ephemeral secrets).
2. Both clients agree on the exact same fingerprint after every rotation.

Figure 13 — Matching Phase 5 Rekey Fingerprints Across Rotations



Dual 60-second rekey rotations showing matching fingerprints on dev (middle) and sanchit (right) at 23:23:58 and 23:24:58, while the server (left) relays opaque ciphertext.

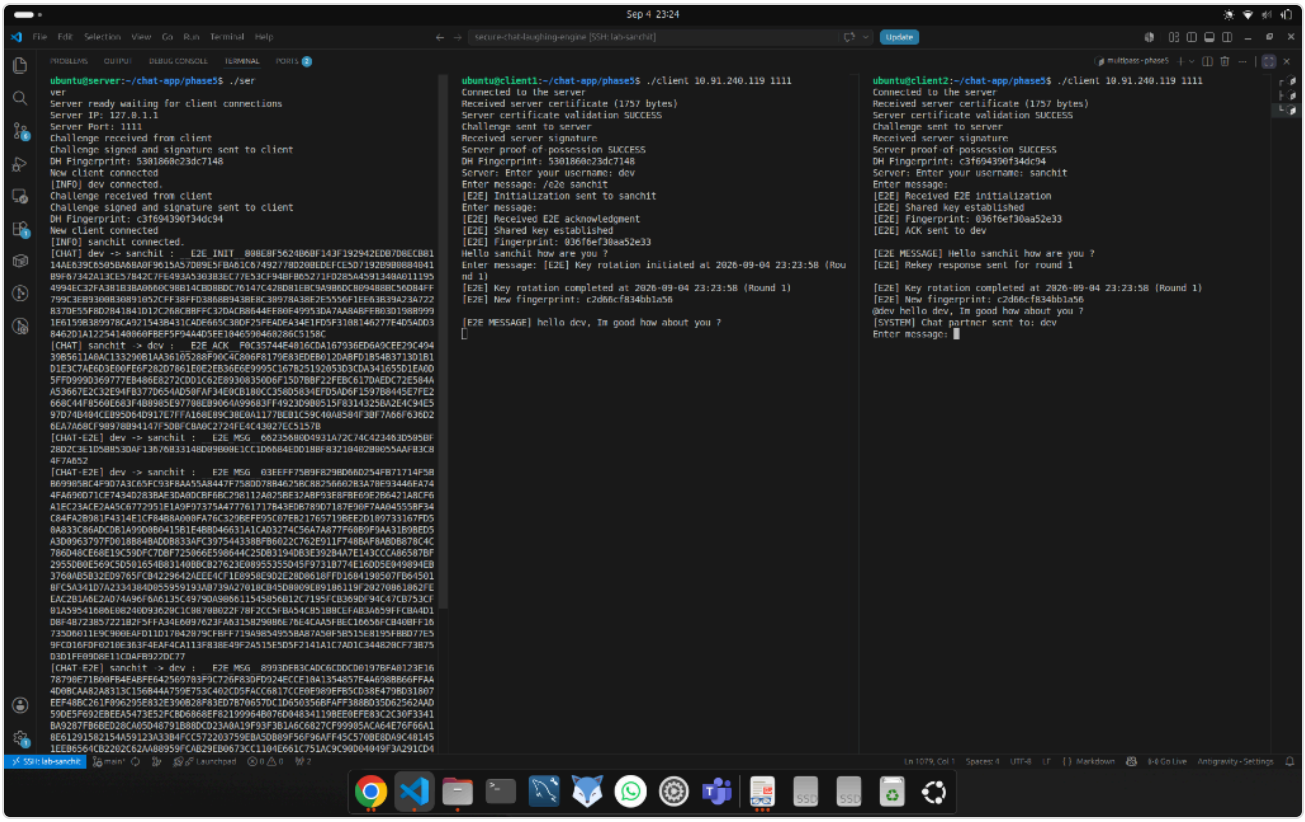
5.2 Post-Rotation Message Delivery

To confirm that ongoing communication was not disrupted by key rotation, a message was sent across the virtual network immediately after the Round 1 rotation completed:

- `sanchit` (on `client2` VM) sent: `"@dev hello dev, Im good how about you ?"`
- `dev` (on `client1` VM) received and decrypted: `"[E2E MESSAGE] hello dev, Im good how about you ?"`

The server log on the `server` VM confirmed that it only relayed opaque ciphertext (`[CHAT-E2E] sanchit -> dev : E2E_MSG 8993DFB3CADC6CDD...`) without accessing message content or key material.

Figure 14 — Successful Post-Rotation Message Delivery Across VMs



Live post-rotation message delivery verified across VMs: client 2 transmits message after Round 1 rekey, client 1 decrypts successfully, and server only relays ciphertext.

The message was decrypted without errors or dropped packets, proving that the live chat session transitions seamlessly across key boundaries over a live network.

---

6. Discussion: Forward Secrecy Security Properties

In terms of attacker capabilities, forward secrecy provides the following concrete security guarantees:

- **What an attacker CAN do if a key is compromised:**
  If an adversary compromises one of the endpoint machines and extracts the active symmetric key K_i, the adversary can only decrypt messages transmitted during that specific 60-second window.
- **What an attacker CANNOT do:**
  1. **Past Traffic (Forward Secrecy):** The adversary cannot decrypt any past messages sent using keys K_1, K_2, ..., K_{i-1}. Because old keys are wiped from memory via `destroyDHE()` and each DH exchange uses fresh random exponents

(a_i, b_i), knowing K_i provides zero mathematical advantage in deriving past shared secrets.

2. **Future Traffic (Post-Compromise Security):** Once the next 60-second rotation occurs, fresh ephemeral exponents are generated. Unless the adversary maintains persistent, continuous code execution on the host, they cannot decrypt subsequent sessions.

Phase 4 alone lacked this guarantee: a compromise of the single session key in Phase 4 exposes the entire communication history of that session. Phase 5 confines the blast radius of any key compromise to a single 60-second time slice.

Appendix A — Generative AI Usage

Some of the prompts that were used during the development of the assignment:

Phase 1 — Baseline Chat Application

- Explain how to implement a one-to-one chat application using TCP sockets in C++.
- Help me understand how the client and server should communicate in a TCP chat application.
- Help me fix this problem in TCP socket chat application.

Phase 2 — Diffie–Hellman and AES-GCM

- Explain how Diffie–Hellman key exchange works and how it can be implemented using OpenSSL BIGNUM.
- Help me implement Diffie–Hellman manually using RFC 3526 Group 14 without using OpenSSL's built-in DH/ECDH APIs.
- How can I derive an AES-256 key from the Diffie–Hellman shared secret using SHA-256?
- Help me implement AES-256-GCM encryption and decryption using OpenSSL.
- Help me implement a MITM proxy to demonstrate the vulnerability in the unauthenticated Diffie–Hellman exchange.
- Help me troubleshoot AES-GCM decryption and tamper detection.

Phase 3 — PKI and Server Authentication

- Explain how to create a Certificate Authority and issue a server certificate using OpenSSL.
- Help me implement server certificate validation in the C++ client using OpenSSL.
- How can I verify that a server possesses the private key corresponding to its certificate?
- Help me implement a challenge-response proof-of-possession mechanism using OpenSSL signatures.
- Help me update the MITM attack so that it fails against certificate validation and proof-of-possession.
- Help me troubleshoot certificate validation and signature verification errors.

Phase 4 — End-to-End Encryption

- Explain how to modify the existing chat application to provide end-to-end encryption between clients while the server only relays encrypted messages.

- Help me implement a separate Diffie–Hellman exchange between two clients while maintaining the existing client-server encrypted connection.
- How should E2E_INIT, E2E_ACK, and E2E_MSG messages be handled?
- Help me troubleshoot why the E2E acknowledgement is not being received by the initiating client.
- Why can simultaneous E2E key exchanges cause the two clients to derive different keys?
- Review my client.cpp and identify the issue with the E2E handshake.
- Can the E2E implementation be fixed without introducing TCP message framing?

Phase 5 — Forward Secrecy via Key Rotation

- How do I implement periodic key renegotiation in a chat application using a background timer thread?
- How to avoid collisions when both clients attempt to initiate key renegotiation at the same time?
- What is the difference between forward secrecy and break-in recovery (post-compromise security)?
- How should old keys be securely erased from memory in OpenSSL (using BN_clear and vector zeroing)?
- How to ensure that chat messages sent during or right before a key rotation are not dropped?

Troubleshooting and Testing

- Help me troubleshoot this OpenSSL compilation error: fatal error: openssl/bn.h: No such file or directory.
- Help me troubleshoot SSH key and host-key issues on the lab machines.
- Help me understand what the Wireshark packet capture demonstrates about confidentiality.
- What evidence should I include in the report to demonstrate that the MITM attack succeeds or fails?
- Review my implementation and suggest what should be tested for each security phase.

Report Preparation

- Help me structure the Network Security assignment report.
- Rewrite my Phase 1 implementation description in a clear technical style.
- Help me explain the security improvements between each phase.
- What screenshots and evidence should be included in the report?
- Help me write the discussion of security properties and limitations.